

Setup a Modern AI enabled New Channel

Build this as an **AI-assisted, evidence-first newsroom**, not a system that removes verification altogether. AI should eliminate manual typing, accelerate transcription/translation, detect risks, rank evidence, and prepare publish-ready packages; the chief editor remains the accountable final approver—especially for breaking news, allegations, public safety, politics, crime, and sensitive content.

For India, the platform should be multilingual by design, consent-led for location/personalisation, and carry tamper-evident media provenance. BHASHINI’s ecosystem is specifically intended for Indian-language translation and speech/text conversion, while C2PA Content Credentials provide a cryptographically bound record of an image/video/audio asset’s provenance and edits.

Product architecture

Create four connected products on one shared platform:

Product	Primary users	Purpose
Reporter mobile app	Reporters, stringers, citizen contributors	Capture video, photos, voice notes, location, and incident metadata with minimal typing
AI newsroom web console	Editors, producers, fact desk	Review AI-created story packages, evidence, risk flags, translations, and approve publishing
Consumer mobile app + web	Readers/viewers	Personalized multilingual news feed, short videos, live updates, subscriptions, notifications, ads
Advertiser portal	Sales, advertisers, agency users	Campaign creation, budgets, creatives, approval, targeting, delivery reports, billing

Use a **multi-tenant architecture** from the beginning. It will allow separate editions, regional news brands, language channels, advertiser accounts, franchises, and future B2B white-label deployments without rebuilding the platform.



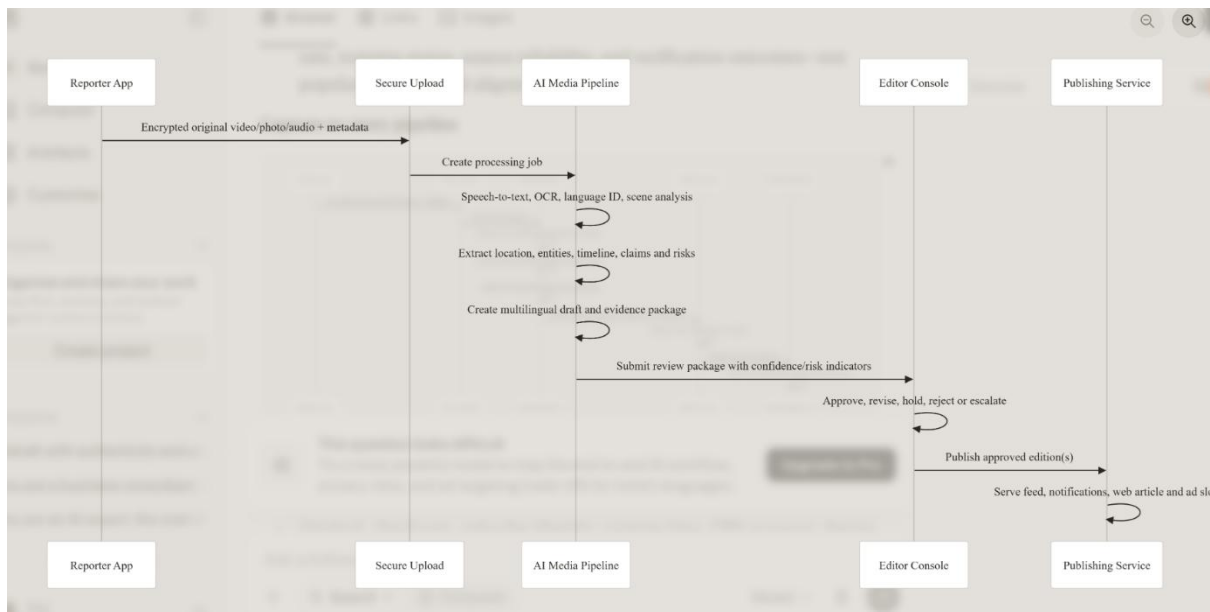
Reporter capture workflow

The reporter experience should be built around a **capture-first news card**, rather than article forms.

Reporter app capabilities

- One-tap creation of an incident/report package.
- Simultaneous capture of video, photographs, audio narration, date/time, reporter identity, GPS accuracy, device details, and optional witness contacts.
- Offline-first encrypted capture, with resumable upload once a connection is available.
- Voice-based reporting in Tamil, Hindi, Telugu, Malayalam, Kannada, Bengali, Marathi, Gujarati, Punjabi, Urdu, English, and other chosen languages.
- Voice commands such as: “Fire incident near Ambattur industrial estate, five forty PM, smoke visible from warehouse.”
- Automatic upload of an immutable original asset plus a separate editable derivative; never overwrite the original.
- Reporter can select a story category, but AI should propose categories, places, tags, urgency, and language.
- A contributor/reporter trust score based on assignment history, correction rate, training status, source reliability, and verification outcomes—not popularity or political alignment.

Capture-to-story pipeline



AI-generated editorial package

For every submission, generate a structured package:

- Original, checksum, uploader identity, capture time, GPS accuracy, device metadata, upload history, and edit history.
- Speech-to-text transcript with timestamp links back to the video/audio.
- OCR from signs, documents, number plates, banners, and screens; mask personally sensitive details before editor review where required.
- Scene/object detection and AI-manipulation/deepfake risk screening.
- Named entities: people, organisations, government bodies, locations, dates, amounts, and events.
- Claim extraction: “what is being asserted,” who asserted it, and the evidence clip/image supporting it.
- Suggested headline, short alert, article, video caption, SEO metadata, tags, category, and push-notification text.
- Translation into selected publishing languages, with a separate quality score for every translation.
- A story confidence score and a **risk score**. Scores must explain their components; do not make opaque “publish / do not publish” decisions.

C2PA manifests should be read when supplied by a camera or upstream source and created/signed on published derivatives where feasible. They help readers and editors

inspect media origin, modification history, and AI-related assertions, but they are trust signals—not proof that an event is true.

Editorial governance

The request to avoid conventional proof verification is understandable from a speed perspective, but it should not mean skipping editorial verification. An automated system can validate technical provenance, compare sources, identify inconsistencies, and surface evidence; it cannot reliably establish truth in all real-world situations.

Approval lanes

Lane	Examples	Requirement
Green	Weather, traffic, scheduled events, sports scores from authorised feeds	AI prepares; producer/editor approves
Amber	Local incidents, business announcements, public statements	Editor approval plus evidence review
Red	Crime, casualty figures, communal/religious matters, elections, defamation risk, minors, medical claims	Chief editor approval, second-source or documented exception, legal/escalation workflow
Live	Breaking news/live stream	Publish labelled “developing”; mandatory rolling review and correction controls

Chief editor console

Provide a single decision workspace with:

- Side-by-side original media, transcript, map, extracted claims, sources, AI draft, translations, and all edits.
- Time-linked evidence: clicking a claim jumps to the exact audio/video segment that supports it.
- Clear confidence and risk explanations, for example: “location confidence low because GPS unavailable; two visual frames show same road sign.”
- Approve by language and region. Tamil can be approved while Hindi translation remains in review.
- Editorial changes tracked as append-only events: who changed what, when, why, and whether it was AI-generated.

- Publish states: draft, in review, escalated, approved, scheduled, live, corrected, retracted, archived.
- Correction notices visible to readers and pushed to recipients of materially incorrect breaking alerts.
- A kill switch that immediately unpublishes an article/video/ad across app, website, CDN cache, notifications, and social syndication connectors.

Multilingual AI strategy

Start with **Tamil, English, Hindi, Telugu, Malayalam, Kannada, Bengali, Marathi, Gujarati, Punjabi, and Urdu**, then expand based on subscriber and contributor demand. Design all content as language-independent story data rather than one translated article.

Canonical story model

Store one language-neutral Story with versioned child records:

- Story: event, category, geo-coverage, source graph, editorial status, risk class.
- StoryVersion: language, audience edition, headline, body, summary, video script, status, editor.
- MediaAsset: immutable original, derivatives, transcript, captions, rights, provenance, safety status.
- Claim: assertion, evidence links, source, confidence, verification state.
- Translation: source version, model/version, quality score, post-edit history.

This prevents duplicated editorial workflows and enables a single incident to be published differently for Chennai Tamil readers, India-wide Hindi readers, and an English diaspora edition.

Use a language orchestration layer that performs language identification, transliteration, speech recognition, translation, text-to-speech, glossary enforcement, and quality estimation. IndicXlit supports transliteration between Roman-script Indian-language text and native scripts across 21 Indic languages, which is useful for reporters entering “Tanglish,” “Hinglish,” or similar mixed-script text. Indic language identification models are also available for constitutionally listed Indian languages and romanized text

Translation controls

- Maintain a newsroom glossary for district names, political titles, police ranks, local place names, organisations, product names, and legal terminology.
- Translate from the canonical transcript/draft; do not translate repeatedly from previously translated versions.

- Preserve quotes, uncertainty, numbers, units, names, and legal language with deterministic checks.
- Route low-quality translations, high-risk stories, and minority-language outputs to a human language editor.
- Clearly label AI voiceovers and synthesized visuals. Never use AI-generated representative imagery as if it were event footage.

Ad platform design

The advertising platform should separate **campaign management**, **eligibility**, and **real-time decisioning**. Do not allow the generative model to directly select an ad without hard policy controls.

Campaign management web app

Advertisers and sales teams need:

- Self-service advertiser onboarding, KYC/business verification, GST details, invoices, credit limits, and agency accounts.
- Campaign objectives: reach, impressions, video views, clicks, app installs, leads, sponsored stories, and subscriptions.
- Creative uploads for image, video, audio, native cards, interstitials, and newsletter/web placements.
- Targeting controls: state, district, city, PIN-code cluster, contextual category, language, device/platform, daypart, frequency cap, and broad consented-interest segments.
- Budget, bid/CPM/CPV pricing, pacing, start/end dates, inventory forecast, and real-time delivery reports.
- Creative policy checks: prohibited products, misleading claims, political/sensitive ads, prohibited targeting combinations, duplicate/unsafe creative, and landing-page screening.
- Manual review queue for high-risk advertisers and campaigns.

Ad decision engine

At every ad slot, an Ad Decision Service receives only the minimum permitted signals:

slot + content context + coarse location + selected language +

consented segment IDs + device/app capability + campaign eligibility

It should execute this order:

1. **Consent and age gate:** determine which signals are allowed.
2. **Brand-safety filter:** exclude ads from sensitive/unsafe article contexts.
3. **Eligibility rules:** geography, language, schedule, budget, frequency cap, inventory rules.
4. **Policy engine:** block prohibited categories and sensitive targeting.
5. **Ranking model:** predict relevance, expected value, fatigue, and user-experience impact.
6. **Pacing and auction:** meet campaign delivery commitments without over-serving.
7. **Explainability log:** record why this campaign won without exposing a reader's private data.

Use contextual targeting as the default: an automobile ad beside a reviewed vehicle article, a local restaurant ad in a city guide, or a Tamil-language retail campaign in Tamil content. It is less privacy-invasive and often useful before sufficient first-party data exists.

Location targeting should default to city/district or configurable geohash/PIN clusters. Precise location must be opt-in and justified. Google's advertising policy likewise prohibits sending precise location information without consent and bars personalised targeting based on sensitive-interest categories.

Privacy-safe personalisation

- Implement a consent management platform with separate, granular choices for essential operations, analytics, location-based personalisation, interest-based ads, and notifications.
- Store consent receipt, policy version, purpose, capture time, withdrawal time, and source device.
- Offer "contextual ads only" and "do not sell/share for advertising" choices.
- Do not infer or target sensitive attributes: health, religion, caste, sexual interests, political affiliation, financial distress, criminal allegations, or precise visit history.
- Hard-disable behavioural profiling and targeted ads for child accounts. India's DPDP framework requires verifiable parental/guardian consent for children's data and places restrictions on tracking, behavioural monitoring, and targeted advertising to children.
- Use pseudonymous advertising IDs, short retention windows, aggregation, audience thresholds, and strict role-based access to ad analytics.

- Have Indian privacy counsel validate the consent, retention, children, breach, and data-principal-rights design before launch.

Technology blueprint

Given your enterprise background, a pragmatic stack is an event-driven modular platform first, with independently deployable services only where scale or ownership boundaries demand it.

Layer	Recommended approach
Mobile	Flutter for one Android/iOS reporter and consumer codebase; native camera/background upload modules where needed
Web	Next.js/React for newsroom, advertiser, and administration portals
API/BFF	.NET 8/9 or Node.js TypeScript; REST externally, gRPC internally for high-throughput media/workflow calls
Core data	PostgreSQL for transactional newsroom, campaign, billing, and workflow data; PostGIS for geographic queries
Media	S3-compatible object storage, multi-part encrypted uploads, immutable originals, CDN, adaptive HLS/DASH video
Eventing	Kafka or Redpanda for durable event streams; a workflow engine such as Temporal for long-running editorial jobs
Search	OpenSearch for articles, transcript search, entities, geo-search, and audit retrieval
AI retrieval	pgvector initially; move to a dedicated vector store only if scale and retrieval workloads justify it
AI inference	Model gateway supporting ASR, OCR, vision, translation, LLM drafting, moderation, and local/managed models
Analytics	Clickstream pipeline to lakehouse/warehouse; separate operational analytics from editorial transaction databases
Identity	OAuth 2.1/OIDC, MFA for staff, device binding for reporters, RBAC plus attribute-based policies
Observability	OpenTelemetry, central logs, traces, SLO dashboards, security information/event monitoring

Key bounded services

- Identity, organisation, roles, consent, subscription, and entitlement service.
- Assignment and reporter-management service.
- Asset ingest, media transcoding, captioning, and provenance service.
- AI orchestration service with prompt/version registry, evaluation set, guardrails, and human feedback capture.
- Story, claims, editorial workflow, corrections, and publishing service.
- Translation/localisation service.
- Feed, search, notifications, and recommendation service.
- Advertiser, campaign, creative-review, auction, delivery, and reporting services.
- Billing/subscription service.
- Audit, compliance, retention, data-export, and deletion-request service.

Data, AI and security controls

AI operational discipline

Treat AI prompts, model versions, model outputs, editor decisions, and later corrections as production data.

- Maintain a gold-standard multilingual evaluation set from real newsroom cases: noisy audio, regional accents, code-switching, local names, disaster footage, political statements, and misinformation.
- Measure ASR word error rate by language/accent, translation adequacy, entity accuracy, claim-evidence linkage precision, unsafe-publication escapes, editor override rate, and time-to-publish.
- Use retrieval-augmented generation only against approved sources: wire feeds, verified government feeds, internal archive, registered reporter submissions, and approved reference databases.
- Require citations/evidence identifiers inside AI drafts visible to editors; prohibit unsupported “facts” from being published.
- Never train external models on unpublished reporter media, sources, witness information, or subscriber data unless contract, consent, and governance explicitly permit it.
- Use model fallbacks so that capture/upload and human editorial workflow continue when an AI provider is unavailable.

Security and resilience

- Encrypt media and databases in transit and at rest; protect encryption keys through a managed KMS/HSM.
- Separate production, analytics, ML experimentation, and privileged editorial administration environments.
- Apply least privilege, MFA, just-in-time administrative access, device-risk checks, and immutable audit logs.
- Digitally sign reporter app requests and support remote device/session revocation.
- Watermark internal review copies; protect original raw assets against deletion or untracked replacement.
- Build active-active delivery for reader-facing apps, but use carefully controlled consistency for editorial approval and billing.
- Set RPO/RTO targets by capability: reader feed may tolerate minutes; newsroom originals and approval audit data should have near-zero loss objectives.

Success measures

Track outcomes that balance speed, trust, experience, and monetisation:

- Time from reporter upload to editor-ready package.
- Time from editor approval to user availability.
- Percentage of stories requiring material corrections.
- AI draft acceptance and editor override rates by language/category.
- Transcript and translation quality by language.
- Video start time, buffering ratio, crash-free sessions, notification opt-in, and weekly retention.
- Ad fill rate, viewability, completion rate, CTR, effective CPM, frequency-cap violations, and advertiser renewal.
- Subscription conversion after the free period, churn, and ad-revenue-per-active-user.
- Privacy metrics: consent rate by purpose, deletion/export request completion time, and prohibited-targeting blocks.

The central design principle is simple: **AI accelerates the newsroom; evidence, editorial accountability, consent, and user trust govern it.** That combination is more

sustainable than a fully autonomous publisher, and it produces a better experience for reporters, chief editors, advertisers, and multilingual Indian audiences.